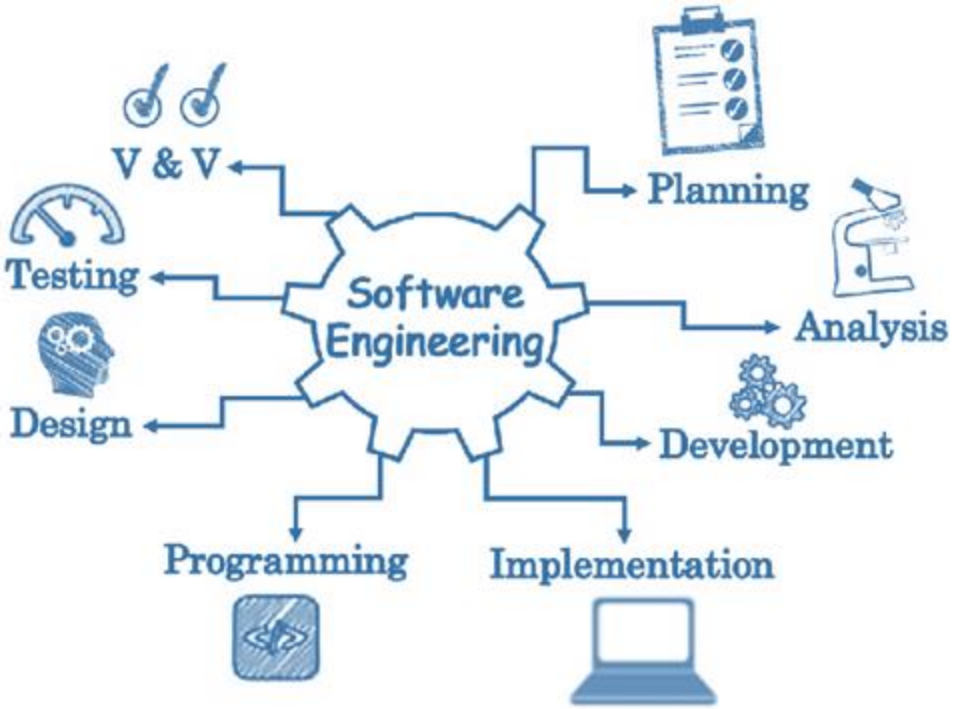
Software Engineering (CMP 519.3)



- Instructor
- Birodh Rijal

birodh.rijal@eemc.edu.np

Software Process Models and Project

3.1 Requirements Elicitation Concepts

- Importance of Stakeholder Involvement
- Challenges in Requirements Gathering

3.2 Detailed Requirements Analysis

- Completeness and Consistency
- Clarity and Correctness
- Realism and Verifiability
- Traceability of Requirements

3.3 Elicitation Activities

- Identifying Actors and Stakeholders
- Developing Scenarios and Use Cases
- Refining Use Cases and User Stories
- Identifying Relationships among Actors and Use Cases

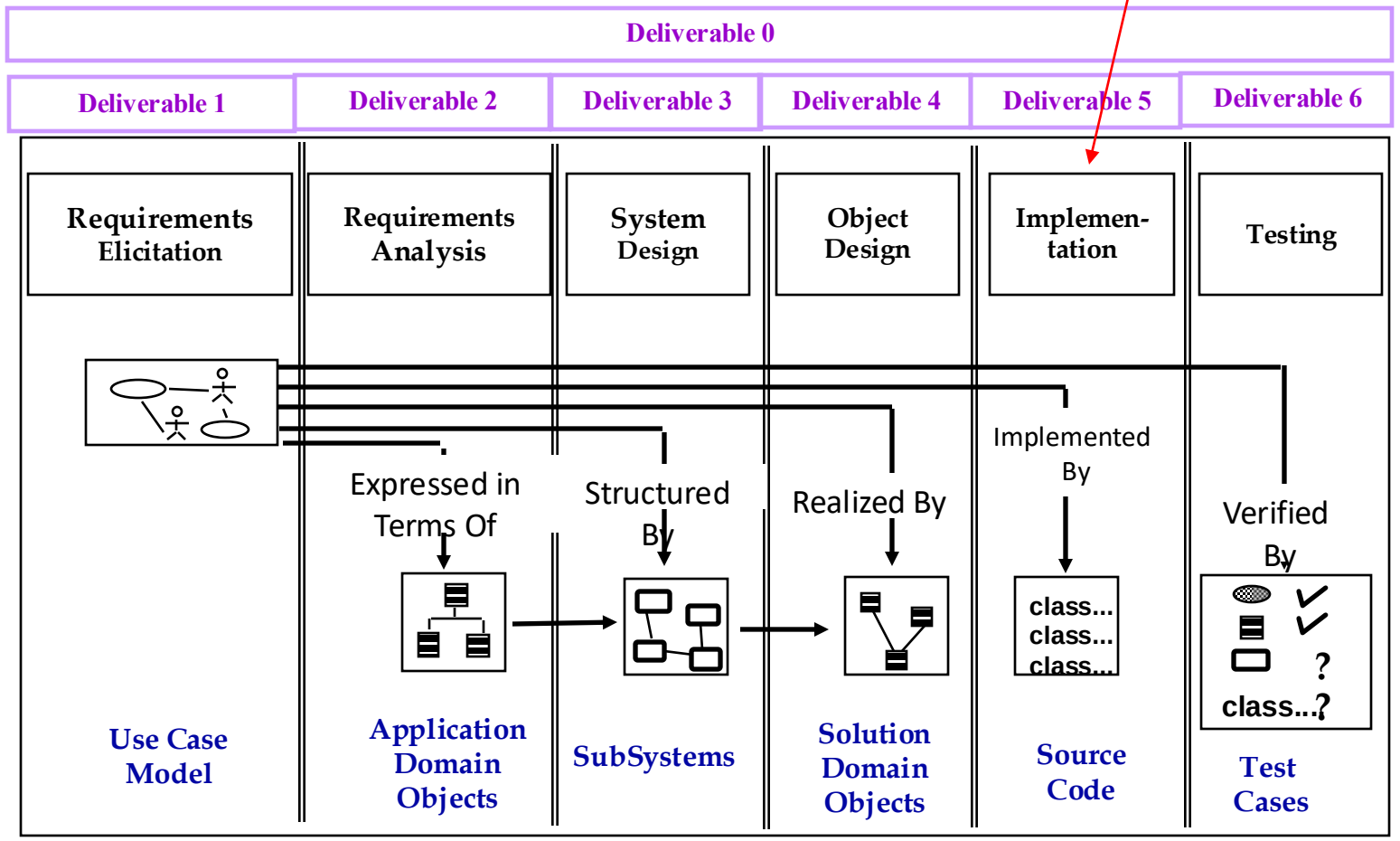
3.4 Managing Requirements

- Maintaining Traceability
- Negotiating Specifications with Clients
- Documenting Requirements

Formalism

Software Lifecycle Activities

The Hacker knows only one activitity



Each activity produces one or more models

Requirement Engineering

- In fact, building software is so compelling that many software developers want to jump right in before they have a clear understanding of what is needed. Which can lead to a failed software project.

- From a software process perspective, requirements engineering is a major software engineering action that begins during the communication activity and continues into the modeling activity.



Requirement Engineering process

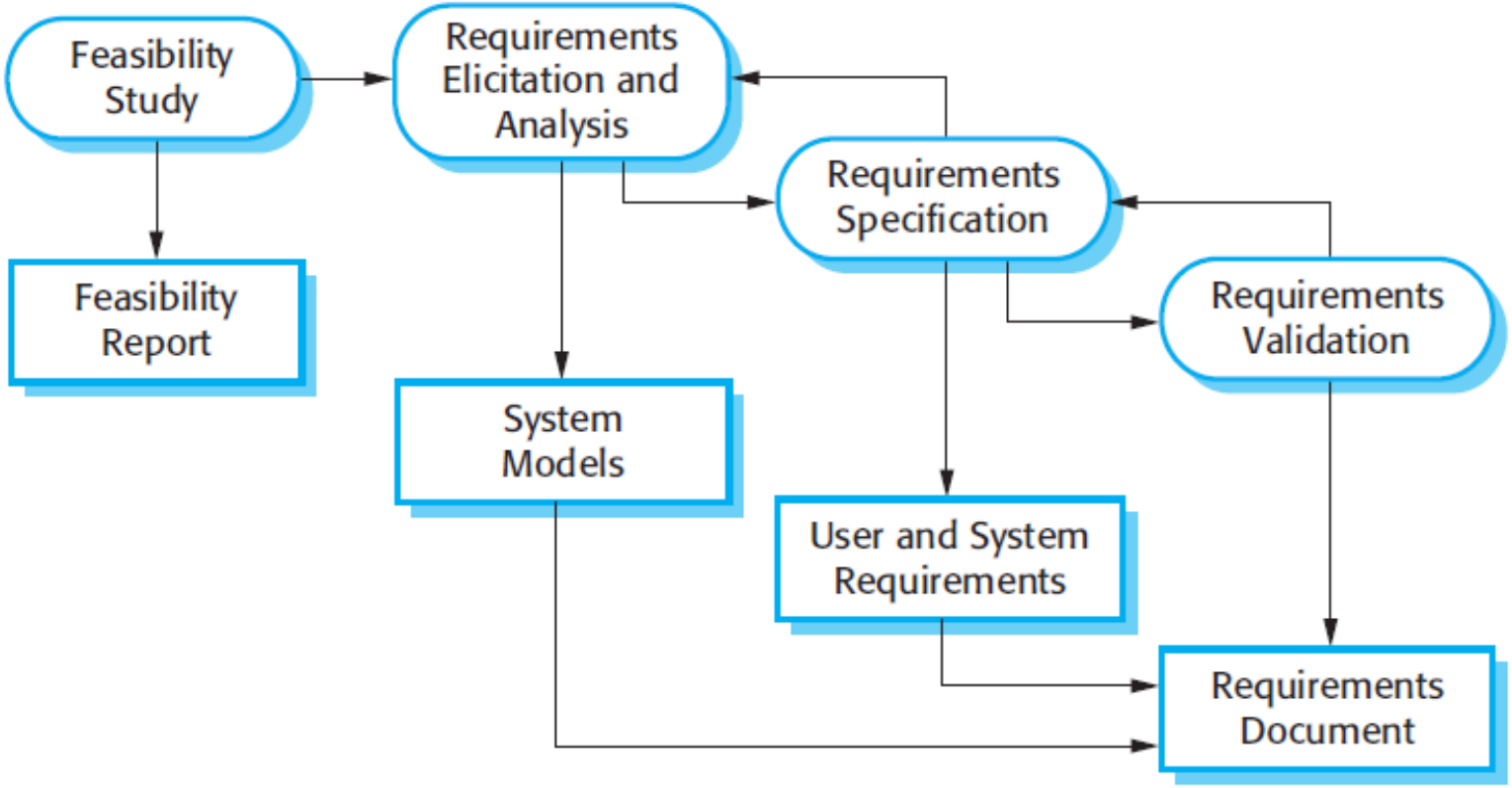


Figure: Requirement Engineering Process

Figure: 4 Agile Values

First Step in Establishing the Requirements: System Identification

- The development of a system is not just done by taking a snapshot of a scene (domain)
- Two questions need to be answered:
  - How can we identify the *purpose* of a system?
  - Crucial is the definition of the *system boundary*: What is inside, what is outside the system?
- The requirements process consists of two activities:
  - Requirements Elicitation:
    - Definition of the system in terms understood by the *customer* ("Problem Description")
  - Requirements Analysis:
    - *Technical* specification of the system in terms understood by the *developer* ("Problem Specification")

Requirements Elicitation Concept

- ***Requirements specification***
  - The client, the developers, and the users identify a problem area and define a system that addresses the problem. Such a definition is called a ***requirements specification***
  - Serves as a contract between the client and the developers.
- ***Requirement Analysis***
  - The requirements specification is structured and formalized during analysis to produce an analysis model
- Both requirements specification and analysis model represent the same information.
- They differ only in the language and notation they use; the requirements specification is written in natural language, whereas the analysis model is usually expressed in a formal or semiformal notation.

Requirements Elicitation Concept

- The requirements specification supports the communication with the ***client*** and ***users***.
- The analysis model supports the communication among ***developers***.
- Given that both models represent the same aspects of the system, requirements elicitation and analysis occur concurrently and iteratively.

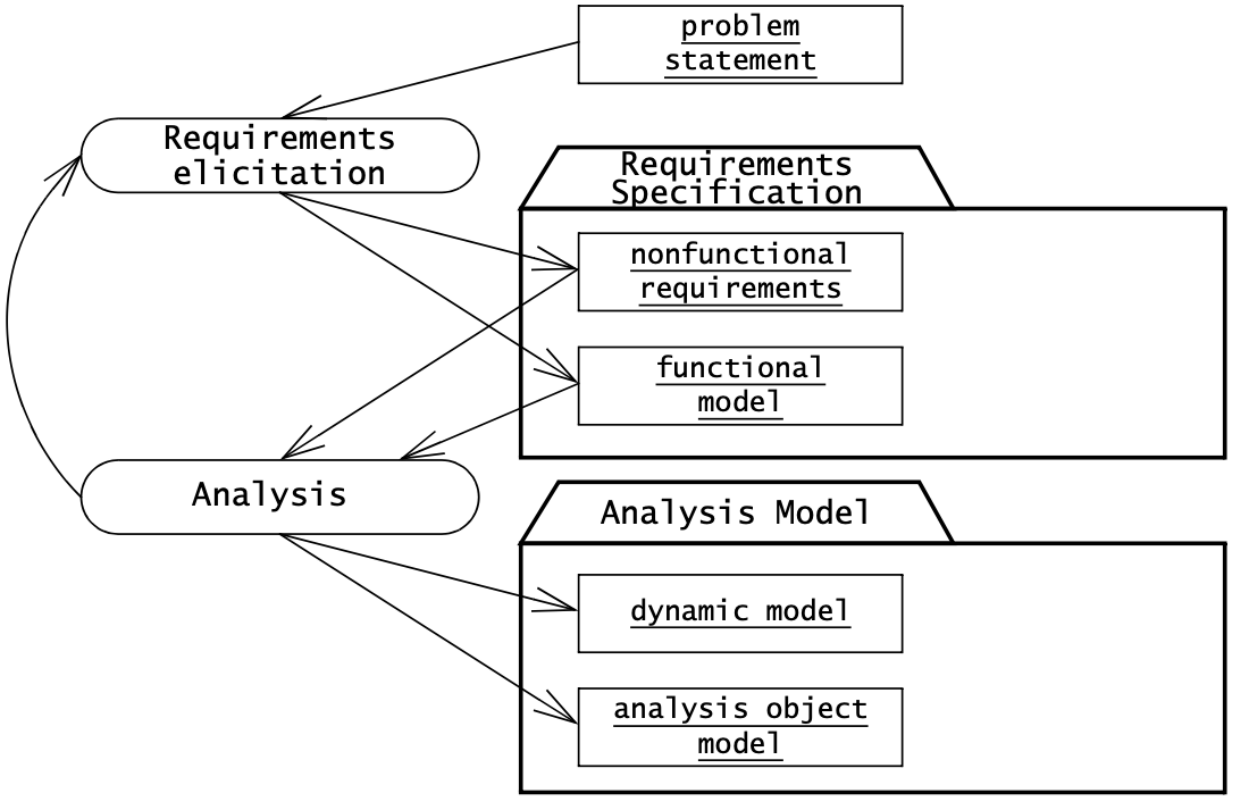


Figure 4-1 Products of requirements elicitation and analysis (UML activity diagram).

Requirements Elicitation Concept

Ingredients of a Problem Statement

- **Current situation**: The Problem to be solved
  - Description of one or more scenarios
- ***Requirements***
  - Functional and Nonfunctional requirements
  - Constraints (“pseudo requirements”)
- ***Project Schedule***
  - Major milestones that involve interaction with the client including deadline for delivery of the system
- ***Target environment***
  - The environment in which the delivered system has to perform a specified set of system tests
- ***Client Acceptance Criteria***
  - Criteria for the system tests

Requirements Elicitation Concept

- Functional Requirements
  - **Functional requirements** describe the interactions between the system and its environment independent of its implementation.
  - The environment includes the user and any other external system with which the system interacts.

SatWatch is a wrist watch that displays the time based on its current location. SatWatch uses GPS satellites (Global Positioning System) to determine its location and internal data structures to convert this location into a time zone.
The information stored in SatWatch and its accuracy measuring time is such that the watch owner never needs to reset the time. SatWatch adjusts the time and date displayed as the watch owner crosses time zones and political boundaries. For this reason, SatWatch has no buttons or controls available to the user.
SatWatch determines its location using GPS satellites and, as such, suffers from the same limitations as all other GPS devices (e.g., inability to determine location at certain times of the day in mountainous regions). During blackout periods, SatWatch assumes that it does not cross a time zone or a political boundary. SatWatch corrects its time zone as soon as a blackout period ends.
SatWatch has a two-line display showing, on the top line, the time (hour, minute, second, time zone) and on the bottom line, the date (day, date, month, year). The display technology used is such that the watch owner can see the time and date even under poor light conditions.
When political boundaries change, the watch owner may upgrade the software of the watch using the WebifyWatch device (provided with the watch) and a personal computer connected to the Internet.

Figure 4-2 Functional requirements for SatWatch.

Requirements Elicitation Concept

- Non-Functional Requirements
  - **Nonfunctional requirements** describe aspects of the system that are not directly related to the functional behavior of the system.
  - Nonfunctional requirements include a broad variety of requirements that apply to many different aspects of the system, from usability to performance.
  - The FURPS+ model2 used by the Unified Process [Jacobson et al., 1999] provides the following categories of nonfunctional requirements:
    - **Functionality**, **Usability**, **Reliability**, **Performance**, and **Supportability**.
  - The FURPS+ model provides additional categories of requirements
    - Implementation requirements
    - Interface requirements
    - Operations requirements
    - Packaging requirements
    - Legal requirements

Requirement Gathering

- Requirements gathering combines elements of problem solving, elaboration, negotiation, and specification.

- To encourage a collaborative, team-oriented approach to requirements gathering, stakeholders work together to identify the problem, propose elements of the solution, negotiate different approaches, and specify a preliminary set of solution requirements.

Collaborative Requirements Gathering

- *Meetings* (either real or virtual) are conducted and attended by both software engineers and other stakeholders.
- *Rules* for preparation and participation are established.
- An *agenda* is suggested that is formal enough to cover all important points but informal enough to encourage the free flow of ideas.
- A "*facilitator*" (can be a customer, a developer, or an outsider) controls the meeting.
- A *"definition mechanism"* (can be worksheets, flip charts, or wall stickers or an electronic bulletin board, chat room, or virtual forum) is used.
- The goal is
  - to identify the problem
  - propose elements of the solution
  - negotiate different approaches, and
  - specify a preliminary set of solution requirements.

Requirements Elicitation Concept

- Very challenging activity
- Requires collaboration of people with different backgrounds
  - ***Users*** with application domain knowledge
  - ***Developer*** with solution domain knowledge (design knowledge, implementation knowledge)
- Bridging the gap between user and developer:
  - *Scenarios:* Example of the use of the system in terms of a series of interactions with between the user and the system
  - *Use cases:* Abstraction that describes a class of scenarios

Challenges in Requirements Gathering

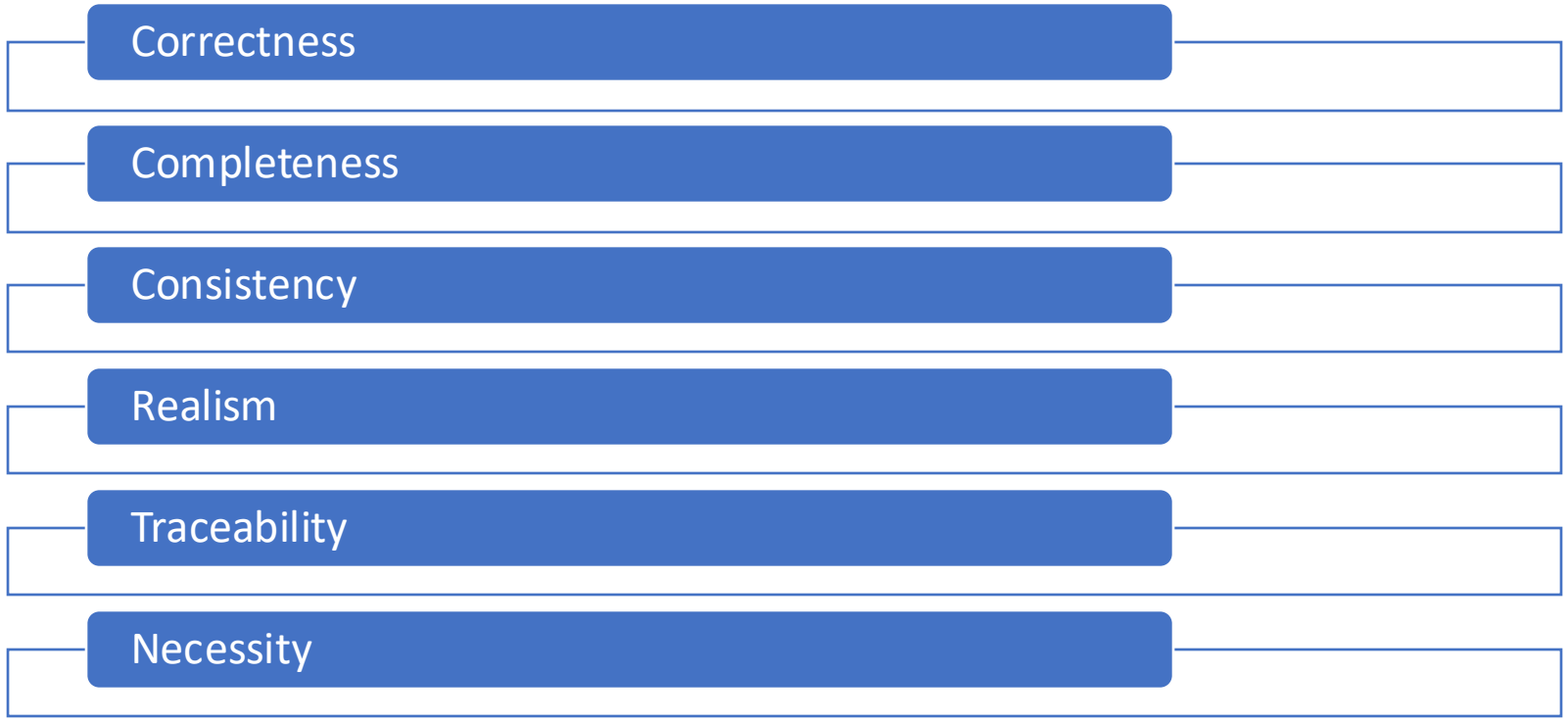
- **Communication barriers**
  - Stakeholders may not have a clear understanding of their requirements, or they may express their requirements using technical jargon or ambiguous language, which can be difficult to interpret.
- **Lack of stakeholder engagement**
  - Some stakeholders may not be interested or invested in the project, and as a result, may not provide the necessary input required for successful requirement gathering.
- **Conflicting requirements**
  - Different stakeholders may have conflicting requirements or priorities, which can make it challenging to develop a cohesive set of requirements that meet everyone's needs.

Challenges in Requirements Gathering

- **Changing requirements**
  - Requirements can change over time, either due to changing business needs or changes in the project scope. This can make it challenging to keep track of all the changes and ensure that the requirements remain aligned with the project goals.
- **Managing expectations**
  - Stakeholders may have unrealistic expectations or expectations that are not feasible given the project constraints, such as time or budget.
- **Resistance to change**
  - Some stakeholders may be resistant to change, and as a result, may be unwilling to provide input or may push back on proposed changes.

Requirement Analysis: Validation

- Requirements validation is a critical step in the development process, usually after requirements engineering or requirements analysis. Also at delivery (client acceptance test).



Requirement Analysis: Validation

- **Complete**—All features of interest are described by requirements.
  - *Example of incompleteness:* The SatWatch specification does not specify the boundary behavior when the user is standing within GPS accuracy limitations of a state's boundary.
  - *Solution:* Add a functional requirement stating that the time depicted by SatWatch should not change more often than once very 5 minutes.
- **Consistent**
  - No two requirements of the specification contradict each other.
  - *Example of inconsistency:* A watch that does not contain any software faults need not provide an upgrade mechanism for downloading new versions of the software.
  - *Solution:* Revise one of the conflicting requirements from the model (e.g., rephrase the requirement about the watch not containing any faults, as it is not verifiable anyway).

Requirement Analysis: Validation

- **Unambiguous**
  - A requirement cannot be interpreted in two mutually exclusive ways.
  - *Example of ambiguity:* The SatWatch specification refers to time zones and political boundaries. Does the SatWatch deal with daylight saving time or not?
    Solution: Clarify the ambiguous concept to select one of the mutually exclusive phenomena (e.g., add a requirement that SatWatch should deal with daylight saving time).
- **Correct**
  - The requirements describe the features of the system and environment of interest to the client and the developer, but do not describe other unintended features.
  - *Example of fault:* There are more than 24 time zones. Several countries and territories (e.g, India) are half an hour ahead of a neighboring time zone.
- **Necessity**
  - Does every requirement contribute to the fulfillment of the goals defined?

The correctness and completeness of a requirements specification are often difficult to establish, especially before the system exists.

Realism, Verifiability, and Traceability

- **Realism**
  - The requirements specification is **realistic** if the system can be implemented within constraints and it can be delivered within the timeframe.

- **Verifiability**
  - The requirements specification is **verifiable** if, once the system is built, repeatable tests can be designed to demonstrate that the system fulfills the requirements specification.
  - ***The product shall have a good user interface*.—Good is not defined. (Non-verifiable)**

- **Traceability**
  - A requirements specification is **traceable** if each requirement can be traced throughout the software development to its corresponding system functions, and if each system function can be traced back to its corresponding set of requirements.
  - When developing tests, traceability enables a tester to assess the coverage of a test case, that is, to identify which requirements are tested and which are not.

Discussion

- Why is it that many software developers don’t pay enough attention to requirements engineering? Are there ever circumstances where you can skip it?

- You have been given the responsibility to elicit requirements from a customer who tells you he is too busy to meet with you. What should you do?

Elicitation Activities

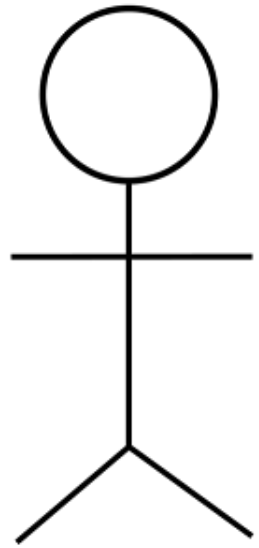
- **Identifying Actors**
- **Identifying Scenarios**
- **Identifying Use Cases**
- **Refining Use Cases**
- **Identifying Relationships Among Actors and Use Cases**

Actor

- Actors represent external entities that interact with the system.
- An actor can be human or an external system.
- Actors are role abstractions and do not necessarily directly map to persons. The same person can fill the role of *FieldOfficer* or *Dispatcher* at different times. However, the functionality they access is substantially different. For that reason, these two roles are modeled as two different actors.

Questions for identifying actors

•Which user groups are supported by the system to perform their work?
•Which user groups execute the system's main functions?
•Which user groups perform secondary functions, such as maintenance and administration?
•With what external hardware or software system will the system interact?



Scenarios

- "A narrative description of what people do and experience as they try to make use of computer systems and applications" [M. Carrol, Scenario-based Design, Wiley, 1995]

- A concrete, focused, informal description of a single feature of the system used by a single actor.

- Scenarios can have many different uses during the software lifecycle
  Requirements Elicitation: **As-is scenario, visionary scenario**
  Client Acceptance Test: **Evaluation scenario**
  System Deployment: **Training scenario.**

Scenarios

Scenario name	*warehouseOnFire*
Participating actor instances	*bob, alice:FieldOfficer* *john:Dispatcher*
Flow of events	1. Bob, driving down main street in his patrol car, notices smoke coming out of a warehouse. His partner, Alice, activates the “Report Emergency” function from her FRIEND laptop. 2. Alice enters the address of the building, a brief description of its location (i.e., northwest corner), and an emergency level. In addition to a fire unit, she requests several paramedic units on the scene, given that the area appears to be relatively busy. She confirms her input and waits for an acknowledgment. 3. John, the *Dispatcher*, is alerted to the emergency by a beep of his workstation. He reviews the information submitted by Alice and acknowledges the report. He allocates a fire unit and two paramedic units to the Incident site and sends their estimated arrival time (ETA) to Alice. 4. Alice receives the acknowledgment and the ETA.

Scenarios

Scenario name	*warehouseOnFire*
Participating actor instances	*bob, alice:FieldOfficer* *john:Dispatcher*
Flow of events	1. Bob, driving down main street in his patrol car, notices smoke coming out of a warehouse. His partner, Alice, activates the "Report Emergency" function from her FRIEND laptop. 2. Alice enters the address of the building, a brief description of its location (i.e., northwest corner), and an emergency level. In addition to a fire unit, she requests several paramedic units on the scene, given that the area appears to be relatively busy. She confirms her input and waits for an acknowledgment. 3. John, the *Dispatcher*, is alerted to the emergency by a beep of his workstation. He reviews the information submitted by Alice and acknowledges the report. He allocates a fire unit and two paramedic units to the Incident site and sends their estimated arrival time (ETA) to Alice. 4. Alice receives the acknowledgment and the ETA.

- Concrete scenario
  - **Describes a single instance of reporting a fire incident.**
  - **Does not describe all possible situations in which a fire can be reported.**
- Participating actors
  - **Bob, Alice and John**

Scenarios

As-is scenario:

- Used in describing a current situation. Usually used in re-engineering projects. The user describes the system.

Visionary scenario:

- Used to describe a future system. Usually used in greenfield engineering and reengineering projects.
- Can often not be done by the user or developer alone

Evaluation scenario:

- User tasks against which the system is to be evaluated.

Training scenario:

- Step by step instructions that guide a novice user through a system

Finding the Scenarios

- Don’t expect the client to be verbal if the system does not exist (greenfield engineering)
- Don’t wait for information even if the system exists
- Engage in a ***dialectic*** approach (evolutionary, incremental engineering)
  - You help the client to formulate the requirements
  - The client helps you to understand the requirements
  - The requirements evolve while the scenarios are being developed

Heuristics for Finding the Scenarios

- Ask yourself or the client the following questions:
  - What are the primary tasks that the system needs to perform?
  - What data will the actor create, store, change, remove or add in the system?
  - What external changes does the system need to know about?
  - What changes or events will the actor of the system need to be informed about?
- However, don't rely on *questionnaires* alone.
- Insist on *task observation* if the system already exists (interface engineering or reengineering)
  - Ask to speak to the end user, not just to the software contractor
  - Expect resistance and try to overcome it

Identifying Use Cases

- A ***scenario*** is an instance of a ***use case***; that is, a use case specifies all possible scenarios for a given piece of functionality.
- Description of a sequence of interactions between a system and external actors
- Developed by Ivar Jacobson

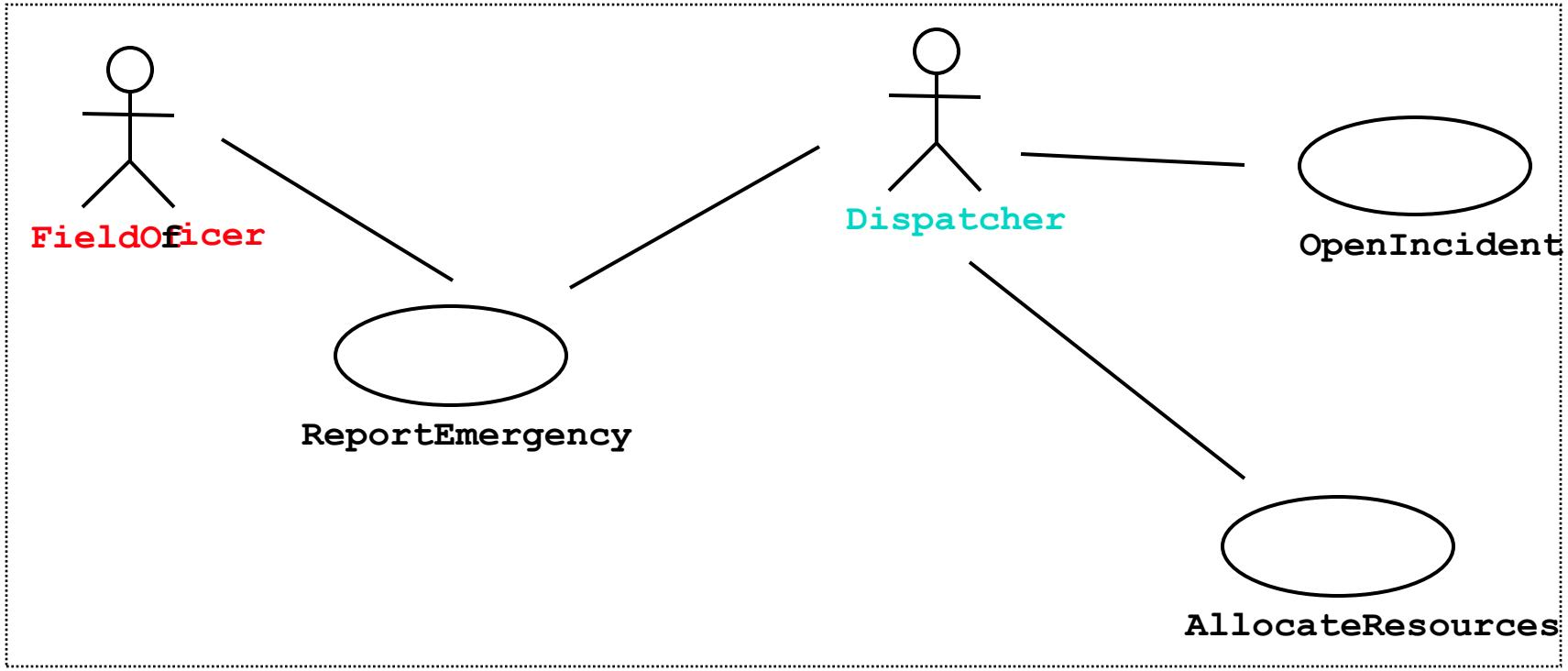
Actors – any agent that interact with the system to achieve a useful goal (e.g., people, other software systems, hardware)

Use case describes a typical sequence of actions that an actor performs in order to complete a given task

- The objective of use case analysis is to model the system
  - from the point of view of how actors interact with this system ……..when trying to achieve their objectives
  - A use case model consists of
    - A set of use cases
    - An optional description or diagram indicating how they are related

Identifying Use Cases

How do we go from scenarios to use cases?



Identifying Use Cases

How do we go from scenarios to use cases?

Name of Use Case
Actors: Description of Actors involved in use case)
Entry condition: "This use case starts when…"
Flow of Events: Free form, informal natural language
Exit condition: "This use cases terminates when…"
Exceptions: Describe what happens if things go wrong
Special Requirements: NFR, Constraints

- Use case name: **ReportEmergency**
- Participating Actors:
  - **Field Officer** (Bob and Alice in the Scenario)
  - **Dispatcher** (John in the Scenario)
- Exceptions:
  - The FieldOfficer is notified immediately if the connection between her terminal and the central is lost.
  - The Dispatcher is notified immediately if the connection between any logged in FieldOfficer and the central is lost.
- Flow of Events: **on next slide.**
- Special Requirements:
  - The FieldOfficer's report is acknowledged within 30 seconds. The selected response arrives no later than 30 seconds after it is sent by the Dispatcher.

Identifying Use Cases

How do we go from scenarios to use cases?

Use Case Example: ReportEmergency

Flow of Events

- The **FieldOfficer** activates the "Report Emergency" function of her terminal. FRIEND responds by presenting a form to the officer.
- The FieldOfficer fills the form, by selecting the emergency level, type, location, and brief description of the situation. The FieldOfficer also describes possible responses to the emergency situation. Once the form is completed, the FieldOfficer submits the form, at which point, the **Dispatcher** is notified.
- The **Dispatcher** reviews the submitted information and creates an Incident in the database by invoking the OpenIncident use case. The Dispatcher selects a response and acknowledges the emergency report.
- The FieldOfficer receives the acknowledgment and the selected response.

Identifying Use Cases

Use case name	`ReportEmergency`
Participating actors	Initiated by `FieldOfficer` Communicates with `Dispatcher`
Flow of events	1. The `FieldOfficer` activates the "Report Emergency" function of her terminal. 2. `FRIEND` responds by presenting a form to the `FieldOfficer`. 3. The `FieldOfficer` completes the form by selecting the emergency level, type, location, and brief description of the situation. The `FieldOfficer` also describes possible responses to the emergency situation. Once the form is completed, the `FieldOfficer` submits the form. 4. `FRIEND` receives the form and notifies the `Dispatcher`. 5. The `Dispatcher` reviews the submitted information and creates an `Incident` in the database by invoking the `OpenIncident` use case. The `Dispatcher` selects a response and acknowledges the report. 6. `FRIEND` displays the acknowledgment and the selected response to the `FieldOfficer`.
Entry condition	• The `FieldOfficer` is logged into `FRIEND`.
Exit conditions	• The `FieldOfficer` has received an acknowledgment and the selected response from the `Dispatcher`, OR • The `FieldOfficer` has received an explanation indicating why the transaction could not be processed.
Quality requirements	• The `FieldOfficer`'s report is acknowledged within 30 seconds. • The selected response arrives no later than 30 seconds after it is sent by the `Dispatcher`.

Figure 4-7 An example of a use case, `ReportEmergency`. Under `ReportEmergency`, the left column denotes actor actions, and the right column denotes system responses.

Simple Use Case Writing Guide

•Use cases should be named with verb phrases. The name of the use case should indicate what the user is trying to accomplish (e.g., ReportEmergency, OpenIncident).
•Actors should be named with noun phrases (e.g., FieldOfficer, Dispatcher, Victim).
•The boundary of the system should be clear. Steps accomplished by the actor and steps accomplished
by the system should be distinguished (e.g., in Figure 4-7, system actions are indented to the right).
•Use case steps in the flow of events should be phrased in the active voice. This makes it explicit who
accomplished the step.
•The causal relationship between successive steps should be clear.
•A use case should describe a complete user transaction (e.g., the ReportEmergency use case
describes all the steps between initiating the emergency reporting and receiving an acknowledgment).
•Exceptions should be described separately.
•A use case should not describe the user interface of the system. This takes away the focus from the
actual steps accomplished by the user and is better addressed with visual mock-ups (e.g., the ReportEmergency only refers to the "Report Emergency" function, not the menu, the button, nor the actual command that corresponds to this function).
•A use case should not exceed two or three pages in length. Otherwise, use include and extend relationships to decompose it.

Use case Scenarios Template

Use Case # [number: name]

GENERAL CHARACTERISTICS	
Intent	[a summary statement of the purpose of the use case]
Scope	[one of: company, name of system under design being considered black box, or another as appropriate] <[owning use case package]>
Level	[one of: business level , system level, or component]
Author	[name of use case author(s)]
Last Update:	[date last updated / change history]
Status	[one of : incomplete, under review, finalized, etc.] [one of: conceptualization, analysis, etc.]
Primary Actor	[role name for the primary actor, <optional description>]
Secondary Actors	[role names of other actors (could be systems) relied upon to accomplish use case]
Preconditions	[what we expect is already the state of the world] <[pointer to precondition object model]>
<Dynamic Preconditions>	<[list of Use Cases # that must have run successfully | scenario diagrams that must be previously run]>
Assumptions	[any assumptions relative to this use case]
Trigger	[the event that starts the use case]
Success Post Condition	[the state of the world upon successful completion] <[pointer to post condition object model]>
Failed Post Condition	[the state of the world if use case abandoned] <[pointer to post condition object model]>
< Models>	<[pointer to Use Case-specific Models, if any]>
Operations Concepts	[a description of the approach(s) to be taken by the implementation of this case] <[pointers to visualizations]>
Overview	[description in words that encompasses all scenarios]

MAIN SUCCESS SCENARIO	
Step	**Action**
S	[description in words of the main success scenario] <[pointer to scenario and or activity diagrams for the main success scenario]>
1	["This use case starts when …" followed by the trigger.]
2	[step description… <"included" use case pointer>]
3	[step description… <"included" use case pointer>]
4	["This use case ends when …" the final step in main success scenario.]

Template of Use case detail

Refining the Use Cases

- The focus of this activity is on completeness and correctness.
- Developers identify functionality not covered by scenarios, and document it by refining use cases or writing new ones.
- Developers describe seldom occurring cases and exception handling as seen by the actors.
- Whereas the initial identification of use cases and actors focused on establishing the boundary of the system, the refinement of use cases yields increasingly more details about the features provided by the system and the constraints associated with them.

Refining the Use Cases

Use case name	`ReportEmergency`
Participating actors	Initiated by `FieldOfficer` Communicates with `Dispatcher`
Flow of events	1. The `FieldOfficer` activates the "Report Emergency" function of her terminal. 2. `FRIEND` responds by presenting a form to the officer. *The form includes an emergency type menu (general emergency, fire, transportation) and location, incident description, resource request, and hazardous material fields.* 3. The `FieldOfficer` completes the form by *specifying minimally the emergency type and description fields*. The `FieldOfficer` may also describe possible responses to the emergency situation *and request specific resources*. Once the form is completed, the `FieldOfficer` submits the form. 4. `FRIEND` receives the form and notifies the `Dispatcher` *by a pop-up dialog*. 5. The `Dispatcher` reviews the submitted information and creates an `Incident` in the database by invoking the `OpenIncident` use case. *All the information contained in the `FieldOfficer`'s form is automatically included in the `Incident`. The `Dispatcher` selects a response by allocating resources to the `Incident` (with the `AllocateResources` use case) and acknowledges the emergency report by sending a short message to the `FieldOfficer`.* 6. `FRIEND` displays the acknowledgment and the selected response to the `FieldOfficer`.
Entry condition	• ...

Figure 4-10 Refined description for the `ReportEmergency` use case. Additions emphasized in *italics*.

- The elements that are manipulated by the system are detailed.
- The low-level sequence of interactions between the actor and the system are specified
- Access rights (which actors can invoke which use cases) are specified.
- Missing exceptions are identified and their handling specified.
- Common functionality among use cases are factored out.

Refining the Use Cases

Heuristics for developing scenarios and use cases

- Use scenarios to communicate with users and to validate functionality.
- First, refine a single scenario to understand the user's assumptions about the system. The user may be familiar with similar systems, in which case, adopting specific user interface conventions would make the system more usable.
- Next, define many not-very-detailed scenarios to define the scope of the system. Validate with the user.
- Use mock-ups as visual support only; user interface design should occur as a separate task after the functionality is sufficiently stable.
- Present the user with multiple and very different alternatives (as opposed to extracting a single alternative from the user). Evaluating different alternatives broadens the user's horizon. Generating different alternatives forces developers to "think outside the box."
- Detail a broad vertical slice when the scope of the system and the user preferences are well understood. Validate with the user.

Identifying Relationships among Actors and Use Cases

- Relationships among actors and use cases enable the developers and users to reduce the complexity of the model and increase its understandability.

- ***Communication relationships between actors and use cases***
- ***Extend relationships between use cases***
- ***Include relationships between use cases***

Identifying Relationships among Actors and Use Cases

- ***Communication relationships between actors and use cases***
  - Communication relationships between actors and use cases represent the flow of information during the use case.
  - The actor who initiates the use case should be distinguished from the other actors with whom the use case communicates.

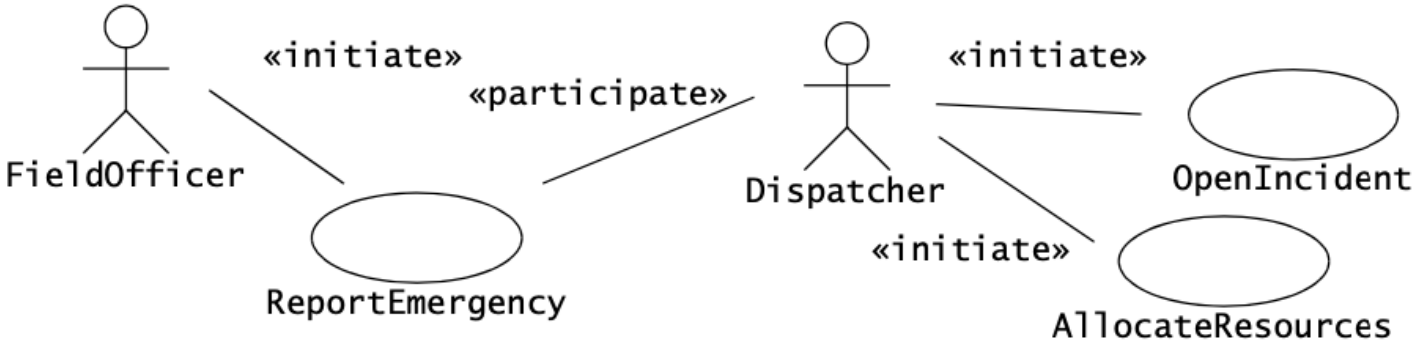


Figure 4-11 Example of communication relationships among actors and use cases in FRIEND (UML use case diagram). The FieldOfficer initiates the ReportEmergency use case, and the Dispatcher initiates the OpenIncident and AllocateResources use cases. FieldOfficers cannot directly open an incident or allocate resources on their own.

Identifying Relationships among Actors and Use Cases

- A use case model consists of use cases and use case associations
  - A use case association is a relationship between use cases
- Important types of use case associations: Include, Extends, Generalization
  - Extends
    - A use case extends another use case
  - Include
    - A use case uses another use case (“functional decomposition”)
  - Generalization
    - An abstract use case has different specializations

Identifying Relationships among Actors and Use Cases

<Extend>> Association for Use Cases

- Problem:
  - The functionality in the original problem statement needs to be extended.
- Solution:
  - An *extend association* from a use case A to a use case B indicates that use case B is an extension of use case A.
- Example:
  - The use case “ReportEmergency” is complete by itself , but can be extended by the use case “ConnectionDown” for a specific scenario in which the user requires to act when there is connection down.

Identifying Relationships among Actors and Use Cases

- ***Extend relationships between use cases***
  - A use case extends another use case if the extended use case may include the behavior of the extension under certain conditions.

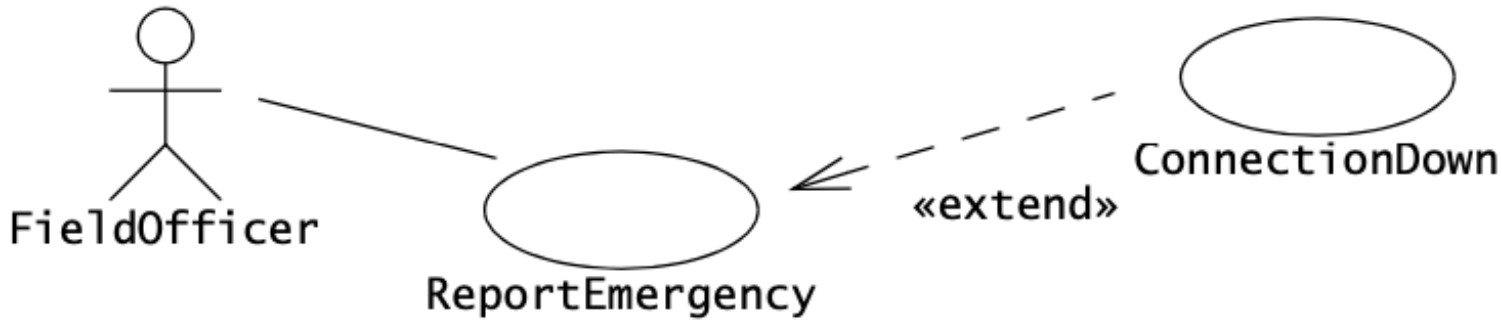


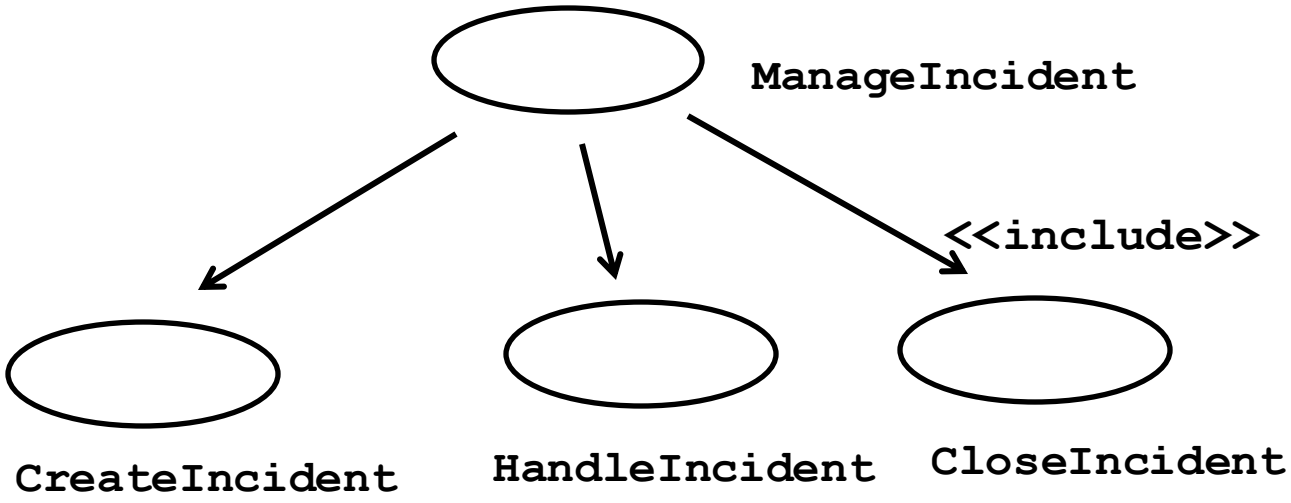
Figure 4-12 Example of use of extend relationship (UML use case diagram). ConnectionDown extends the ReportEmergency use case. The ReportEmergency use case becomes shorter and solely focused on emergency reporting.

Identifying Relationships among Actors and Use Cases

<<Include>>: Functional Decomposition

- Problem:
  - A function in the original problem statement is too complex to be solvable immediately
- Solution:
  - Describe the function as the aggregation of a set of simpler functions. The associated use case is decomposed into smaller use cases

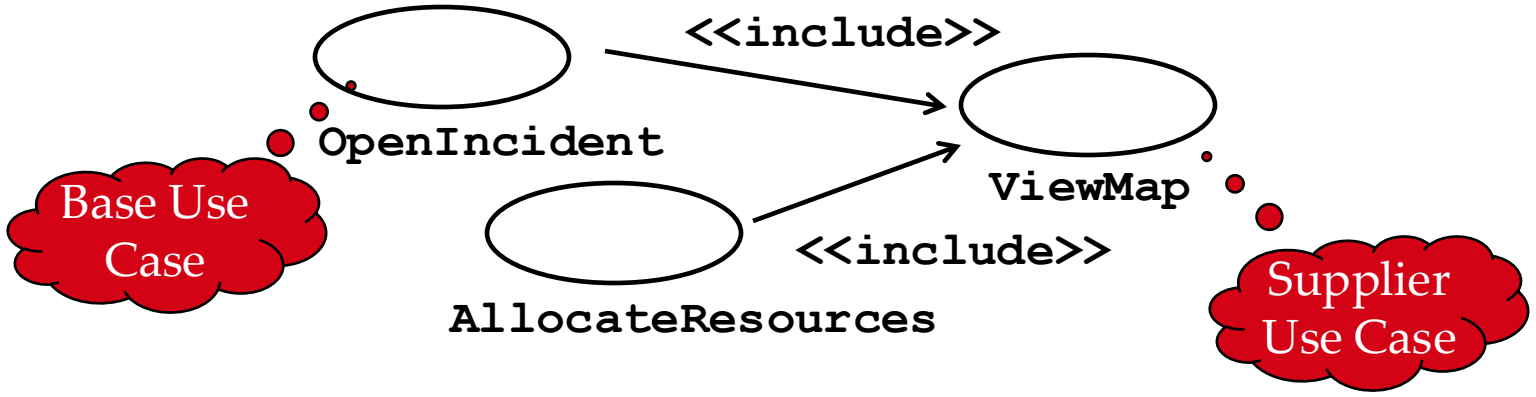
Redundancies among use cases can be factored out using include relationships.



Identifying Relationships among Actors and Use Cases

<<Include>>: Reuse of Existing Functionality

- Problem:
  - There are already existing functions. How can we *reuse* them?
- Solution:
  - The *include association* from a use case A to a use case B indicates that an instance of the use case A performs all the behavior described in the use case B ("A delegates to B")



Example:

The use case "ViewMap" describes behavior that can be used by the use case "OpenIncident" ("ViewMap" is factored out)

Extend versus include relationships

- The main distinction between these constructs is the direction of the relationship.
- For include relationships, the event triggering the target (i.e., included) use case is described in the flow of event of the source use case.
- For extend relationships, the event triggering the source (i.e., extending) use case is described in the source use case as a precondition.

Identifying Relationships among Actors and Use Cases

- **Heuristics for extend and include relationships**
  - Use extend relationships for exceptional, optional, or seldom-occurring behavior. An example of seldom-occurring behavior is the breakdown of a resource (e.g., a fire truck). An example of optional behavior is the notification of nearby resources responding to an unrelated incident.
  - Use include relationships for behavior that is shared across two or more use cases.
  - However, use discretion when applying the above two heuristics and do not overstructure the use
  - case model. A few longer use cases (e.g., two pages long) are easier to understand and review than many short ones (e.g., ten lines long).

Identifying Relationships among Actors and Use Cases

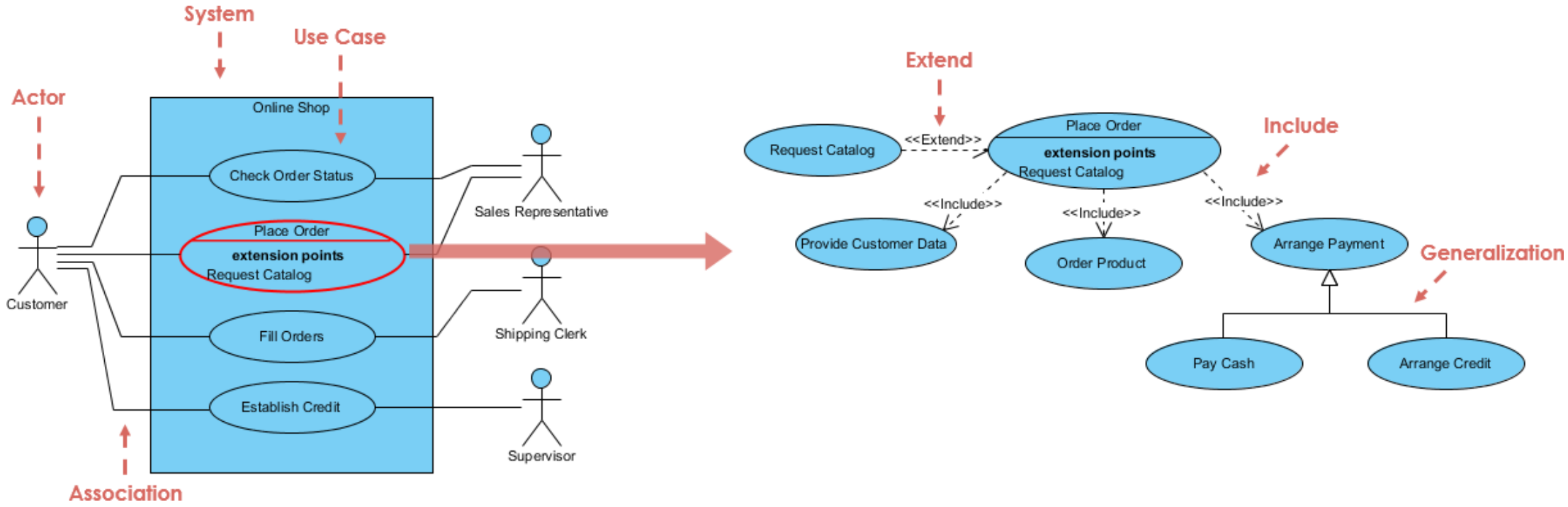


Figure: A use case diagram example consists of four use cases: place order, check status, fill orders and establish credit. Order can be placed directly by customer or a salesperson and they can also check the order status later on. Shipping clerk is responsible for filling the order and the supervise can establish the credit level for customer according the past record and financial information submitted by the customer. Now let us refine the main use case – place order.

Summary

- The requirements process consists of requirements elicitation and analysis.
- The requirements elicitation activity is different for:
  - Greenfield Engineering, Reengineering, Interface Engineering
- Scenarios:
  - Great way to establish communication with client
  - Different types of scenarios: As-Is, visionary, evaluation and training
  - Use cases: Abstraction of scenarios
- Pure functional decomposition is bad:
  - Leads to unmaintainable code
- Pure object identification is bad:
  - May lead to wrong objects, wrong attributes, wrong methods
- The key to successful analysis:
  - Start with use cases and then find the participating objects
  - If somebody asks “What is this?”, do not answer right away. Return the question or observe the end user: “What is it used for?”

Exercise

- Consider your watch as a system and set the time 2 minutes ahead. Write down each interaction between you and your watch as a scenario. Record all interactions, including any feedback the watch provides you.

R= Scenario name *setWatch2MinutesAhead*
Participating actor instances *allen:WatchOwner*
Flow of events

1. The *WatchOwner* presses both Watch buttons simultaneously.
2. The *Watch* enters “set time” mode and indicates this by blinking the hour digits.
3. The *WatchOwner* presses the left button once.
4. The *Watch* stops blinking the hour digits and starts blinking the minutes.
5. The *WatchOwner* presses the right button twice.
6. The *Watch* increments the minutes by two.
7. The *WatchOwner* presses both buttons simultaneously.
8. The *Watch* stops blinking.

Managing Requirements

Requirement management

- Requirements management is a set of techniques for documenting, analyzing, prioritizing and agreeing on requirements so that engineering teams always have current and approved requirements.
- Clear, concise, error-free requirements help engineering teams detect errors early, reducing project cost and risk.
- Requirements can be managed using documents, however, complex systems or products in highly regulated industries mitigate risk by using trusted requirements management tools.

- Negotiating Specifications with Clients: Joint Application Design
- Maintaining Traceability
  Documenting Requirements Elicitation

Requirement management

- Negotiating Specifications with Clients: **Joint Application Design (JAD)**

- Intensive group-oriented requirements determination technique.
- Team members meet in isolation for an extended period of time.
- Highly focused.
- Resource intensive.
- The outcome of the workshop, the final JAD document, is a complete requirements specification document that includes definitions of data elements, work flows, and interface screens.
- Started by IBM in 1970s.
- JAD has been used by IBM and other companies. JAD leverages group dynamics to improve communication among participants and to accelerate consensus. At the end of a JAD session, developers are more knowledgeable of user needs, and users are more knowledgeable of development trade-offs.

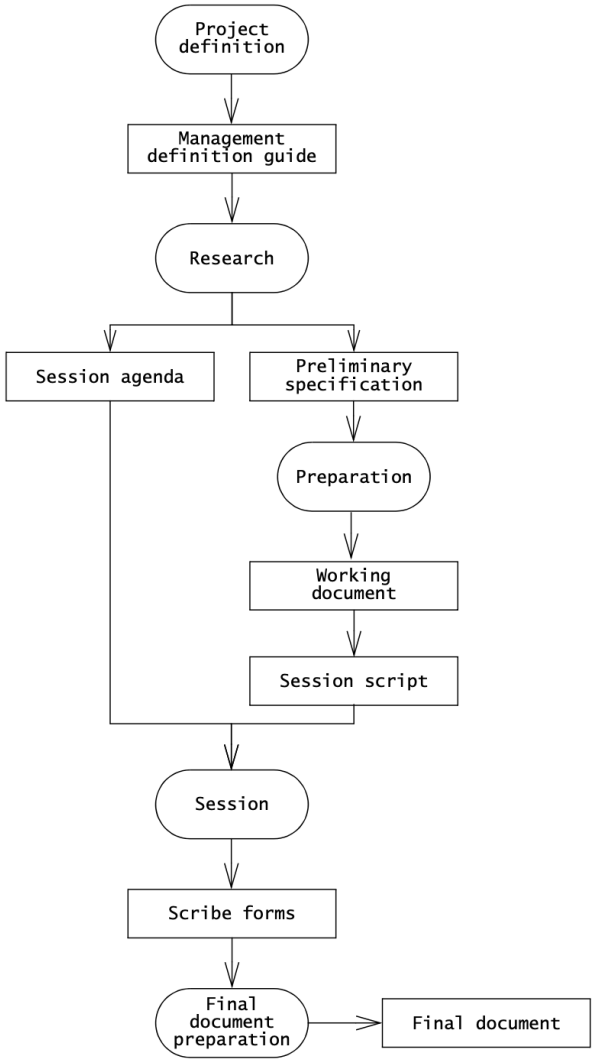


Figure 4-15 Activities of JAD (UML activity diagram). The heart of JAD is the `Session` activity during which all stakeholders design and agree to a requirements specification. The activities prior to the `Session` maximize its efficiency. The production of the final document captures the decisions made during the `Session`.

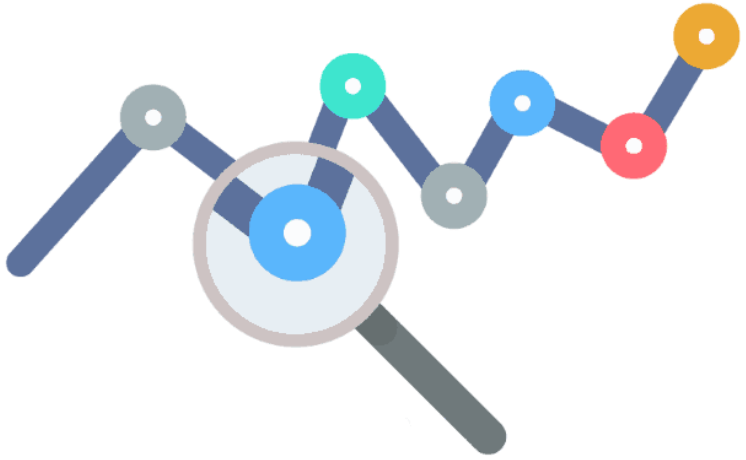
Requirement management

- **Joint Application Design (JAD) : 5 Activities**

- ***Project definition.*** During this activity, the JAD facilitator interviews the project manager and the client to determine the objectives and the scope of the project. The findings from the interviews are collected in the *Management Definition Guide*.
- ***Research***. During this activity, the JAD facilitator interviews present and future users, gathers information about the application domain, and describes a first set of high-level use cases. The JAD facilitator also starts a list of problems to be addressed during the session. The results of this activity are a *Session Agenda* and a *Preliminary Specification* listing work flow and system information.
- ***Preparation***. During this activity, the JAD facilitator prepares the session. The JAD facilitator creates a *Working Document*, which is the first draft of the final document, an agenda for the session, and any overhead slides or flip charts representing information gathered during the Research activity. The JAD facilitator also selects a team composed of the client, the project manager, selected users, and developers. All stakeholders are represented, and the participants are able to make binding decisions.
- ***Session***. During this activity, the JAD facilitator guides the team in creating the requirements specification. A JAD session lasts for 3 to 5 days. The team defines and agrees on the scenarios, use cases, and user interface mock-ups. All decisions are documented by a scribe.
- ***Final document.*** The JAD facilitator prepares the *Final Document*, revising the working document to include all decisions made during the session. The Final Document represents a complete specification of the system agreed on during the session. The Final Document is distributed to the session participants for review. The participants then attend a 1- to 2-hour meeting to discuss the reviews and finalize the document.

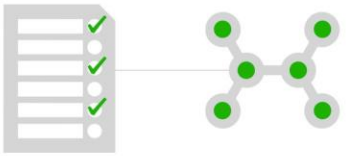
Maintaining Traceability

- Traceability is the ability to follow the **life of a requirement**.
- Full **history of changes** in requirements so you can respond quickly to audits. Your team can see what changed, who changed it and when it changed
- This includes tracing where the requirements came from (e.g., who originated it, which client need does it address) to which aspects of the system and the project it affects (e.g., which components realize the requirement, which test case checks its realization).
- Traceability enables **developers** to show that the system is complete, **testers** to show that the system complies with its requirements, designers to record the rationale behind the system, and maintainers to assess the impact of change.



Maintaining Traceability

- The simplest approach to maintaining traceability is to use ***cross-references*** among documents, models, and code artifacts.
- Each individual element (e.g., requirement, component, class, operation, test case) is identified by a ***unique number***.
- Dependencies are then documented manually as a textual cross-reference containing the number of the source element and the number of the target element.



Maintaining Traceability

Sno	Req ID	Req Desc	TC ID	TC Desc	Test Design	Test Designer	UAT Test Req?	Test Execution			Defects?	Defect ID	Defect Status	Req Coverage Status
								Test Env	UAT Env	Prod Env				
1	Req01	Login to the Application	TC01	Login with Invalid Username and valid password	Completed	XYZ	No	Passed	No Run	No Run	None	None	N/A	Partial
2			TC02	Login with Valid Username and invalid password	Completed	YZA	No	Passed	No Run	No Run	None	None	N/A	Partial
3			TC03	Login with valid credentials	Completed	XYZ	Yes	Passed	Passed	No Run	Yes	DFCT001	Test OK	Partial

Table: requirement traceability matrix

Documenting Requirements Elicitation

- The results of the requirements elicitation and the analysis activities are documented in the **Requirements Analysis Document (RAD)**.
- This document completely describes the system in terms of ***functional*** and ***nonfunctional*** requirements.
- The audience for the RAD includes the ***client***, ***the users***, ***the project management, the system analysts*** (i.e., the developers who participate in the requirements), and ***the system designers*** (i.e., the developers who participate in the system design).

Documenting Requirements Elicitation

Requirements Analysis Document

1. Introduction
   - 1.1 Purpose of the system
   - 1.2 Scope of the system
   - 1.3 Objectives and success criteria of the project
   - 1.4 Definitions, acronyms, and abbreviations
   - 1.5 References
   - 1.6 Overview
2. Current system
3. Proposed system
   - 3.1 Overview
   - 3.2 Functional requirements
   - 3.3 Nonfunctional requirements
     - 3.3.1 Usability
     - 3.3.2 Reliability
     - 3.3.3 Performance
     - 3.3.4 Supportability
     - 3.3.5 Implementation
     - 3.3.6 Interface
     - 3.3.7 Packaging
     - 3.3.8 Legal
   - 3.4 System models
     - 3.4.1 Scenarios
     - 3.4.2 Use case model
     - *3.4.3 Object model*
     - *3.4.4 Dynamic model*
     - 3.4.5 User interface—navigational paths and screen mock-ups
4. Glossary

Figure 4-16 Outline of the Requirements Analysis Document (RAD). Sections in *italics* are completed during analysis (see next chapter).

Assignment

Read the case study: ARENA Case Study,
Page No: 153-167
Object-Oriented Software Engineering Using UML, Patterns, and JavaTM
Third Edition